

SQL

Q.1 Difference Between DBMS and RDBMS

Xin chào và chào mừng các bạn quay trở lại.

Trong buổi học hôm nay, chúng ta sẽ đi sâu vào vai trò quan trọng của SQL trong việc quản lý và phân tích dữ liệu.

Có thể bạn sẽ bất ngờ khi biết rằng hầu như tất cả các công ty lớn như Netflix, Meta, Uber, Google, Microsoft và nhiều công ty khác đều dựa vào SQL.

Ở một mức độ nào đó, SQL đã trở thành kỹ năng thiết yếu cho rất nhiều vị trí, dù bạn là lập trình viên Python, tester, web developer hay software developer, bởi vì SQL giúp bạn kết nối cơ sở dữ liệu với sản phẩm.

Nếu bạn làm việc trong các vai trò thiên về dữ liệu như data analyst, data engineer hoặc data scientist, thì SQL là một kỹ năng bắt buộc phải có để tương tác với cơ sở dữ liệu một cách hiệu quả.

Với nhu cầu ngày càng tăng đối với những chuyên gia có khả năng quản lý và phân tích dữ liệu, việc hiểu rõ SQL trở nên quan trọng hơn bao giờ hết.

Một nền tảng SQL vững chắc sẽ giúp bạn tự tin hơn khi tham gia phỏng vấn và tạo được ấn tượng tốt.

Trong video này, tôi sẽ hướng dẫn bạn 15 câu hỏi phỏng vấn SQL, giúp bạn hiểu rõ hơn những gì có thể gặp trong buổi phỏng vấn.

Vậy không nói nhiều nữa, hãy bắt đầu với câu hỏi đầu tiên.

Câu hỏi đầu tiên: Sự khác biệt giữa DBMS và RDBMS là gì?

Trước hết, nói về DBMS (Database Management System – Hệ quản lý cơ sở dữ liệu).

Đúng như tên gọi, DBMS là cách cơ bản để lưu trữ dữ liệu dưới dạng các tệp (file).

Dữ liệu ở đây được lưu trong các file riêng lẻ, không có mối liên kết giữa chúng.

Ví dụ, hãy tưởng tượng bạn đang học trong một lớp học và giáo viên lưu thông tin của bạn như số báo danh, tên, địa chỉ, số điện thoại vào những file riêng biệt, và không có sự liên kết nào giữa các file đó.

Khi muốn tìm thông tin của một học sinh cụ thể, bạn phải lần lượt kiểm tra từng file một, điều này rất tốn thời gian và công sức.

Ngược lại, RDBMS (Relational Database Management System – Hệ quản lý cơ sở dữ liệu quan hệ) có mối quan hệ giữa các dữ liệu.

Ví dụ như MySQL, bạn có thể dùng nó để tạo bảng và lưu trữ dữ liệu.

Vẫn với ví dụ về học sinh, bạn có thể tạo một bảng gồm các cột như số báo danh, tên và địa chỉ, và các dữ liệu này được liên kết với nhau.

Nhờ vậy, khi bạn muốn tìm kiếm, bạn chỉ cần thực hiện một câu truy vấn, và toàn bộ dữ liệu liên quan đến học sinh đó sẽ được trả về.

Tóm lại, RDBMS giúp việc tạo mối quan hệ giữa các mẫu dữ liệu trở nên dễ dàng hơn.

Điều này cho phép truy xuất dữ liệu nhanh hơn, đặc biệt khi bạn làm việc với tập dữ liệu lớn.

RDBMS cũng giảm sự dư thừa dữ liệu, vì mỗi học sinh chỉ có một bản ghi duy nhất.

Ngoài ra, nó còn được thiết kế để hỗ trợ nhiều người dùng, với mức độ bảo mật cao, và được tối ưu cho khối lượng dữ liệu lớn.

Chúng ta hãy chuyển sang câu hỏi tiếp theo.

Q.2 Primary Key and Foreign Key

Primary key và foreign key là gì?

Chắc hẳn tất cả chúng ta đều từng học ở trường hoặc đại học, và mỗi người đều có số báo danh hoặc mã đăng ký, những mã này là duy nhất cho từng cá nhân, và không có hai người nào trùng số báo danh.

Vì vậy, ta có thể nói rằng số báo danh đóng vai trò là primary key dùng để xác định duy nhất mỗi dòng (row) trong một bảng.

Primary key được dùng để đảm bảo mỗi dòng dữ liệu là khác biệt và không thể bị trùng lặp.

Ngoài ra, primary key không cho phép giá trị rỗng (NULL).

Bạn đã bao giờ thấy một học sinh không có số báo danh trong lớp chưa? Chắc là chưa, đúng không?

Điều này giúp bạn dễ dàng xác định thông tin của từng học sinh mà không gây nhầm lẫn.

Bây giờ, trong lớp học, bạn không chỉ học một môn, ví dụ như Vật lý, Toán, Khoa học, v.v.

Để kết nối bảng thông tin học sinh (gồm tên, địa chỉ, ...) với bảng môn học (thể hiện các môn mà học sinh đang học), bạn sẽ cần một khái niệm gọi là foreign key.

Trong trường hợp này, chúng ta sẽ sử dụng số báo danh để liên kết giữa hai bảng, và nó sẽ đóng vai trò là foreign key.

Foreign key này sẽ tham chiếu ngược lại primary key trong bảng học sinh, từ đó tạo ra mối liên kết giữa hai bảng.

Nói một cách đơn giản, foreign key giúp bạn kết nối dữ liệu giữa các bảng, đảm bảo rằng các bản ghi trong một bảng có thể tham chiếu đến thông tin liên quan ở bảng khác.

Tiếp theo, chúng ta sẽ chuyển sang câu hỏi số 3.

Q.3 Constraints and Their Types

Constraint là gì và các loại của chúng.

Chắc hẳn tất cả các bạn đều đã từng ít nhất một lần điền Google Form, đúng không? Bạn có để ý rằng có những trường bắt buộc phải điền, có những trường không bắt buộc nên bạn có thể để trống, hoặc có những trường chỉ chấp nhận dữ liệu trong một phạm vi nhất định không? Vậy những thứ đó là gì? Chúng được gọi là constraint.

Trong SQL, constraint giống như các quy tắc mà bạn thiết lập cho dữ liệu trong bảng. Chúng giúp giữ cho dữ liệu chính xác và đáng tin cậy.

Nói về các constraint trong SQL:

Constraint đầu tiên là NOT NULL. Đúng như tên gọi, một cột không được phép để trống.

Ví dụ, nếu bạn tạo một schema với cột role number được đặt là NOT NULL, và nếu bạn cố gắng thêm một sinh viên mà không có role number, cơ sở dữ liệu sẽ không cho phép. Điều đó có nghĩa là bắt buộc phải có giá trị cho trường đó.

Constraint thứ hai là UNIQUE. Constraint này đảm bảo rằng không có hai giá trị nào trong một cột là giống nhau.

Giả sử bạn có một cột email trong bảng sinh viên, thì mỗi email phải là duy nhất. Điều này giống như việc nói rằng mỗi sinh viên phải có email riêng biệt.

Tiếp theo là constraint thứ ba, PRIMARY KEY.

Constraint này kết hợp cả NOT NULL và UNIQUE, nghĩa là chỉ có một trường duy nhất được dùng để xác định duy nhất mỗi bản ghi. Như đã nói ở câu hỏi trước, role number đóng vai trò là khóa chính.

Constraint thứ tư là FOREIGN KEY.

Constraint này dùng để liên kết các bảng với nhau. Ví dụ, nếu bạn có một bảng khác liệt kê các môn học, thì khóa ngoại có thể kết nối bảng sinh viên với bảng môn học, liên kết mỗi sinh viên với các môn học của họ.

Constraint thứ năm là CHECK.

Bạn muốn đặt ra một điều kiện nào đó thì constraint này dành cho bạn. Ví dụ, nếu trong bảng của bạn có cột tuổi bầu cử chỉ chấp nhận các giá trị trên 18, thì CHECK có thể áp dụng quy tắc đó cho cột tuổi, và nếu bạn cố gắng thêm một người trẻ hơn, cơ sở dữ liệu sẽ chặn lại.

Cuối cùng là DEFAULT.

Constraint này đóng vai trò quan trọng khi bạn muốn bản ghi có giá trị mặc định nếu không được cung cấp giá trị. Ví dụ, nếu bạn đặt giá trị mặc định là India cho cột quốc gia trong bảng, thì khi bạn không chỉ định quốc gia, hệ thống sẽ tự động điền India.

Tiếp theo, chúng ta chuyển sang câu hỏi tiếp theo.

Q.4 DDL and DML Commands

Câu hỏi tiếp theo là: hãy giải thích các lệnh DDL và DML trong SQL.

Khi chúng ta làm việc với cơ sở dữ liệu hoặc tạo bảng như bảng employee, thì những gì chúng ta thực sự đang làm thường là thiết lập cơ sở dữ liệu, thay đổi dữ liệu của nhân viên hoặc cập nhật các bản ghi của nhân viên, và đó là lúc DDL (Data Definition Language) và DML (Data Manipulation Language) xuất hiện.

Trước tiên là các lệnh DDL.

Nói về DDL, nó giống như công cụ nền tảng của cơ sở dữ liệu. Những lệnh này giúp chúng ta định nghĩa cấu trúc của cơ sở dữ liệu bằng cách tạo, thay đổi hoặc xóa các bảng và các đối tượng khác.

Ví dụ, với lệnh CREATE, chúng ta có thể tạo một bảng mới, như CREATE TABLE employee sẽ giúp tạo ra bảng employee.

Với ALTER, chúng ta có thể chỉnh sửa bảng sau này, chẳng hạn nếu cần thêm một cột mới vào bảng employee.

Thậm chí, việc xóa toàn bộ một bảng cũng có thể thực hiện bằng lệnh DROP.

Vì vậy, có thể nói rằng các lệnh DDL không làm việc với dữ liệu thực tế, mà chỉ làm việc với cấu trúc của cơ sở dữ liệu, và cấu trúc ở đây có nghĩa là cấu trúc tổng thể của bảng.

Bây giờ chuyển sang các lệnh DML.

DML là tất cả về việc làm việc với dữ liệu thực tế. Khi đã có bảng, chúng ta sử dụng DML để chèn, cập nhật hoặc xóa dữ liệu.

Ví dụ, nếu bạn muốn thêm một dòng dữ liệu mới, bạn sẽ dùng INSERT; để thay đổi dữ liệu, bạn sẽ dùng UPDATE; và để xóa dữ liệu cụ thể, DELETE là cách để thực hiện.

Những lệnh này giúp giữ cho dữ liệu trong bảng luôn được cập nhật và dễ quản lý.

Ví dụ, để thêm một bản ghi nhân viên mới, bạn có thể viết một câu lệnh như:

INSERT INTO employee (name, position) VALUES ('Joe', 'Manager').

Tóm lại, các lệnh DDL giống như việc thiết lập và tổ chức ngôi nhà (tức là cấu trúc), còn các lệnh DML giống như cách chúng ta quản lý mọi thứ bên trong ngôi nhà, tức là chỉnh dữ liệu.

Tiếp theo là câu hỏi số năm.

Q.5 Difference Between Delete, Drop, and Truncate

Sự khác nhau giữa các câu lệnh DELETE, DROP và TRUNCATE là gì?

Tất cả chúng ta đều từng có một kệ sách ở nhà để đựng sách giáo khoa, tiểu thuyết hoặc thậm chí là truyện tranh.

Bây giờ, hãy coi kệ sách như một bảng trong cơ sở dữ liệu, và mỗi cuốn sách đại diện cho một dòng dữ liệu.

Nếu bạn muốn xóa dữ liệu khỏi kệ sách này, bạn có ba lựa chọn: DELETE, DROP và TRUNCATE.

Dưới đây là sự khác nhau giữa chúng:

Đầu tiên là DELETE. DELETE giống như việc loại bỏ một cuốn sách cụ thể khỏi kệ dựa trên một điều kiện nào đó, ví dụ như xóa tất cả các cuốn sách có bìa màu đỏ, hoặc xóa sách ở ngăn trên cùng.

DELETE cho phép bạn chọn chính xác những gì cần xóa.

Ngoài ra, nếu bạn đổi ý, bạn có thể rollback để khôi phục lại dữ liệu đã xóa trong hầu hết các trường hợp.

Thứ hai là TRUNCATE. TRUNCATE sẽ xóa nhanh toàn bộ sách khỏi kệ, khiến kệ trống rỗng nhưng vẫn giữ nguyên kệ.

Điều này có nghĩa là dữ liệu bị xóa, nhưng cấu trúc bảng không bị xóa.

Lệnh này thực hiện rất nhanh và không cho phép rollback, tức là khi đã xóa thì không thể hoàn tác.

TRUNCATE phù hợp khi bạn không cần dữ liệu nữa nhưng vẫn muốn giữ cấu trúc bảng để sử dụng trong tương lai.

Thứ ba là DROP. DROP giống như việc dỡ bỏ toàn bộ kệ sách và mang nó ra khỏi phòng.

Tất cả sách trên kệ đều biến mất và bạn không còn lại gì ngoài khoảng trống rỗng.

DROP là vĩnh viễn và không thể hoàn tác, vì vậy chỉ nên dùng khi bạn chắc chắn rằng bạn sẽ không cần bảng hoặc dữ liệu đó nữa trong tương lai.

Q.6 Group By Vs Order By Clause

Hãy tưởng tượng bạn có một bảng, ví dụ như trên màn hình, chứa đầy dữ liệu nhân viên — một danh sách các nhân viên trong công ty, mỗi người có thông tin như phòng ban và mức lương. Bây giờ bạn có hai nhiệm vụ:

Thứ nhất, bạn muốn nhóm nhân viên theo phòng ban để xem mỗi phòng có bao nhiêu người làm việc hoặc mức lương trung bình của từng phòng ban là bao nhiêu.

Thứ hai, bạn muốn sắp xếp danh sách nhân viên theo lương, có thể từ cao xuống thấp hoặc ngược lại.

Hai nhiệm vụ này cần những công cụ khác nhau trong SQL. Công cụ thứ nhất là GROUP BY, và công cụ thứ hai là ORDER BY.

GROUP BY giống như việc chia dữ liệu vào các “thùng”. Ví dụ, bạn gom tất cả nhân viên thuộc cùng một phòng ban vào một nhóm, để bạn có thể dễ dàng tính tổng hoặc giá trị trung bình.

Vì vậy, GROUP BY thường được dùng cùng với các hàm tổng hợp như SUM, AVG, COUNT để tính toán những thứ như: tổng lương của từng phòng ban, lương trung bình của mỗi phòng, hoặc số lượng nhân viên làm việc trong từng phòng ban.

Theo cách này, thay vì xem từng dòng dữ liệu riêng lẻ, bạn sẽ có cái nhìn tổng hợp cho mỗi phòng ban.

Ví dụ, khi tôi viết: `SELECT department, AVG(salary) FROM employee GROUP BY department;`

Câu lệnh này sẽ nhóm các phòng ban như HR, Sales, IT, rồi hiển thị mức lương trung bình tương ứng cho từng phòng.

Tiếp theo là `ORDER BY`, mệnh đề này giống như việc sắp xếp các dòng dữ liệu theo một thứ tự nhất định. Nó giống như việc sắp xếp tên theo thứ tự bảng chữ cái hoặc xếp hạng nhân viên theo mức lương.

Khi bạn dùng `ORDER BY`, bạn đang sắp xếp toàn bộ bảng dựa trên một hoặc nhiều cột, ví dụ như sắp xếp lương từ cao xuống thấp. Bạn có thể sắp xếp theo tăng dần (ASC) hoặc giảm dần (DESC).

Ví dụ: `SELECT * FROM employee ORDER BY salary DESC;`

Với bảng nhân viên gồm tên, phòng ban và lương, câu lệnh này sẽ trả về dữ liệu đã được sắp xếp theo lương từ cao đến thấp.

Tóm lại, `GROUP BY` dùng để nhóm dữ liệu và tạo các bản tóm tắt có ý nghĩa, còn `ORDER BY` dùng để sắp xếp dữ liệu theo thứ tự mà bạn muốn xem.

Tiếp theo, chúng ta chuyển sang câu hỏi số bảy.

Q.7 Difference Between Where And Having Clauses

Sự khác nhau giữa mệnh đề `WHERE` và `HAVING`.

Mệnh đề `WHERE` giống như việc lọc dữ liệu trước khi nhóm.

Hãy tưởng tượng `WHERE` giống như người gác cổng ở cổng khu dân cư. Trước khi bất kỳ ai được vào bên trong — trong trường hợp này là trước khi dữ liệu được `GROUP BY` — người gác cổng sẽ kiểm tra điều kiện và chỉ cho phép những người phù hợp đi vào.

Vậy khi nào dùng `WHERE`?

Bạn dùng `WHERE` để lọc từng dòng dữ liệu riêng lẻ dựa trên các điều kiện cụ thể như tuổi hoặc tên.

Ví dụ, nếu bạn muốn tìm những sinh viên từ 18 tuổi trở lên, bạn sẽ viết:
`SELECT name, age FROM student WHERE age >= 18;`

Câu lệnh này sẽ loại bỏ những sinh viên không đáp ứng điều kiện tuổi trước khi xảy ra bất kỳ quá trình nhóm dữ liệu nào.

Còn `HAVING` thì sao?

HAVING giống như lọc dữ liệu sau khi đã nhóm.

Hãy hình dung HAVING như ban giám khảo trong một chương trình tìm kiếm tài năng. Sau khi các thí sinh đã biểu diễn và được nhóm theo điểm số, ban giám khảo sẽ xem xét các nhóm đó và quyết định nhóm nào đạt tiêu chí.

Vậy khi nào dùng HAVING?

Bạn dùng HAVING khi muốn lọc các nhóm được tạo ra bởi GROUP BY, dựa trên kết quả của các hàm tổng hợp như COUNT hoặc SUM.

Ví dụ, nếu bạn muốn xem số lượng sinh viên trong từng độ tuổi, đồng thời chỉ hiển thị những độ tuổi có hơn một sinh viên, bạn sẽ viết:

```
SELECT age, COUNT(roll_number) AS number_of_students FROM student GROUP BY age HAVING COUNT(roll_number) > 1;
```

Kết quả trả về sẽ chỉ hiển thị những nhóm tuổi có số lượng sinh viên lớn hơn 1.

Tiếp theo, chúng ta chuyển sang câu hỏi số tám.

Q.8 Aggregate Function with Example

Các hàm tổng hợp (aggregate functions) trong SQL là gì và bạn có thể đưa ra ví dụ không?

Hãy tưởng tượng bạn có một giỏ trái cây lớn và bạn muốn biết một vài thông tin về chúng, chẳng hạn như: bạn có bao nhiêu trái, trọng lượng trung bình là bao nhiêu, trái nặng nhất hay nhẹ nhất là trái nào.

Các hàm tổng hợp trong SQL cũng giống như những công cụ giúp bạn thực hiện các tác vụ đó trên dữ liệu trong cơ sở dữ liệu, tương tự như cách bạn làm với giỏ trái cây.

Hãy cùng nói về các hàm tổng hợp này bằng cách thực hiện một số phép toán trên dữ liệu. Bạn có thể thấy bảng hiển thị trên màn hình.

Trước hết là COUNT.

Hàm này đếm số lượng dòng hoặc các giá trị không NULL trong một cột.

Ví dụ, COUNT(*) sẽ cho bạn tổng số bản ghi, trong trường hợp này là 6, vì có 6 nhân viên đang làm việc.

Tiếp theo là SUM.

Hàm này cộng tất cả các giá trị trong một cột dạng số.

Ví dụ, khi bạn viết SUM(salary), nó sẽ cho bạn tổng lương, trong trường hợp này là 3120.

Thứ ba là AVG.

Hàm này tính giá trị trung bình của một cột số.

Ví dụ, AVG(salary) được tính bằng tổng lương chia cho tổng số nhân viên.

Thứ tư là MIN.

Hàm này tìm giá trị nhỏ nhất trong một cột số.

Ví dụ, MIN(salary) sẽ cho ra mức lương thấp nhất, là 43.

Thứ năm là MAX.

Hàm này tìm giá trị lớn nhất trong một cột số.

Ví dụ, MAX(salary) sẽ trả về mức lương cao nhất, trong trường hợp này là 82.

Đó là tất cả về các hàm tổng hợp.

Tiếp theo, chúng ta chuyển sang câu hỏi số chín.

Q.9 Indexing and Clustered Indexing

Giải thích indexing (chỉ mục) trong SQL và clustered index là gì.

Hãy tưởng tượng bạn có một cuốn sách nhưng không có số trang. Chỉ nghĩ thôi cũng thấy việc tìm một chủ đề sẽ khó khăn thế nào khi không có mục lục hay số trang. Tương tự như vậy, hãy coi cơ sở dữ liệu như một đồng giấy khổng lồ chưa được sắp xếp và không có chỉ mục. Nếu bạn muốn tìm một tờ giấy cụ thể, ví dụ như nhân viên có ID 1120, bạn sẽ phải lật từng trang một, điều này rất chậm.

Indexing (chỉ mục) giúp giải quyết vấn đề này. Nó giống như việc tổ chức các tờ giấy theo một trật tự nhất định, giúp việc tìm kiếm trở nên nhanh chóng hơn.

Bây giờ hãy nghĩ đến clustered index. Clustered index giống như việc sắp xếp toàn bộ các tờ giấy theo đúng thứ tự, ví dụ sắp xếp theo employee ID từ nhỏ đến lớn, như bạn thấy trên màn hình. Ở đây, không chỉ đơn giản là gắn nhãn cho các phần, mà là chính dữ liệu cũng được sắp xếp theo thứ tự đó.

Trong hình minh họa trên màn hình, khi chúng ta tìm employee ID 1120, cơ sở dữ liệu sẽ sử dụng clustered index để bỏ qua những phần lớn không phù hợp, nhanh chóng xác định được khoảng từ 1100 đến 1200 và nhảy thẳng đến thư mục đó.

Nhờ vậy, indexing giúp giảm đáng kể số bước tìm kiếm, làm cho việc truy xuất dữ liệu trở nên nhanh và hiệu quả hơn.

Tiếp theo, chúng ta chuyển sang câu hỏi số 10.

Q.10 Normalisation and Different Types of Normal Forms

Normalization (chuẩn hóa) là gì và hãy giải thích các dạng chuẩn khác nhau.

Chuẩn hóa trong cơ sở dữ liệu hoạt động bằng cách tổ chức dữ liệu một cách hiệu quả, giảm thiểu sự dư thừa (tức là thông tin bị lặp lại) và ngăn ngừa các vấn đề khi thêm, xóa hoặc cập nhật bản ghi.

Vậy tại sao normalization lại quan trọng?

Thứ nhất, nó giảm sự trùng lặp dữ liệu. Cũng giống như bạn không muốn có nhiều bản sao của cùng một cuốn sách, trong cơ sở dữ liệu, chuẩn hóa giúp tránh việc lưu cùng một thông tin nhiều lần.

Thứ hai, nó ngăn ngừa các bất thường (anomalies), tức là hạn chế lỗi khi thêm, xóa hoặc cập nhật dữ liệu.

Bây giờ hãy nói về các dạng chuẩn (normal forms).

Đầu tiên là dạng chuẩn thứ nhất (First Normal Form – 1NF).

Trong 1NF, mỗi ô trong bảng chỉ được chứa một giá trị duy nhất, và mỗi cột phải có tên riêng biệt. Điều này đảm bảo không có giá trị lặp trong cùng một ô và giúp việc truy xuất dữ liệu trở nên đơn giản hơn.

Ví dụ, trong bảng thông tin sinh viên, mỗi người có thể có nhiều sở thích. Thay vì liệt kê tất cả sở thích trong một ô, chúng nên được tách thành các dòng riêng biệt, thay vì gộp chung như bạn thấy trên màn hình.

Tiếp theo là dạng chuẩn thứ hai (Second Normal Form – 2NF).

Ở 2NF, tất cả các thuộc tính không phải khóa phải phụ thuộc hoàn toàn vào khóa chính. Nói đơn giản, mỗi mẫu dữ liệu trong bảng phải liên quan trực tiếp đến định danh chính (ví dụ như mã sinh viên), chứ không phụ thuộc vào một thuộc tính khác.

Ví dụ, trong bảng sinh viên, nếu bạn có một cột lưu lớp học và một cột lưu tên giáo viên, thì tên giáo viên phụ thuộc vào lớp học, chứ không phụ thuộc trực tiếp vào mã sinh viên.

Thứ ba là dạng chuẩn thứ ba (Third Normal Form – 3NF).

Hãy tưởng tượng bạn sắp xếp đồ chơi sao cho không có loại đồ chơi nào chồng chéo lên nhau. Tương tự, trong 3NF, mỗi thuộc tính không phải khóa phải độc lập với các thuộc tính không phải khóa khác. Điều này có nghĩa là không có thông tin nào trong bảng được suy ra từ một thông tin không phải khóa khác.

Ví dụ, nếu lớp học của sinh viên gắn với một giáo viên cụ thể, thì thông tin đó không nên trộn chung trong bảng sinh viên, mà nên được tách ra và tổ chức trong một bảng riêng.

Cuối cùng là Boyce–Codd Normal Form (BCNF).

Trong BCNF, mọi định danh (determinant) — tức là một thuộc tính có thể xác định các thuộc tính khác — đều phải là một khóa ứng viên (candidate key). Điều này đảm bảo rằng mọi phụ thuộc đều dựa trên các định danh duy nhất.

Ví dụ, nếu cả mã sinh viên và lớp học đều có thể được coi là định danh, thì mỗi định danh này phải xác định duy nhất dữ liệu trong các cột của nó, mà không bị chồng chéo.

Đó là toàn bộ nội dung về các dạng chuẩn hóa.

Tiếp theo, chúng ta chuyển sang câu hỏi số 11.

Q.11 Union and Union All Operator in SQL

UNION và UNION ALL có nghĩa là gì.

Vậy hãy cùng phân tích toán tử UNION và UNION ALL trong SQL theo cách thật dễ hiểu, thông qua một ví dụ đơn giản.

Hãy tưởng tượng cơ sở dữ liệu SQL giống như danh sách bạn bè của bạn từ hai nơi khác nhau.

Danh sách A gồm bạn bè từ trường học, trong đó có Alice, Bob và Charlie.

Danh sách B gồm bạn bè từ nơi làm việc, trong đó có Bob, David và Eva.

Bây giờ bạn tổ chức một bữa tiệc và muốn tạo một danh sách chung những người sẽ tham dự.

Vậy UNION làm gì?

Toán tử UNION sẽ kết hợp cả hai danh sách thành một, nhưng có một điểm quan trọng: nó loại bỏ các giá trị trùng lặp. Nếu một người xuất hiện trong cả hai danh sách, họ chỉ xuất hiện một lần trong danh sách cuối cùng.

Vì vậy, danh sách khách mời cuối cùng sẽ là: Alice, Bob, Charlie, David và Eva.

Trong SQL, khi sử dụng UNION, câu lệnh sẽ trông như sau:

```
SELECT name FROM school_friends_table UNION SELECT name FROM  
work_friends_table
```

Vậy còn UNION ALL thì sao?

Giả sử bạn muốn bao gồm tất cả mọi người trong cả hai danh sách, kể cả những người bị trùng lặp. Đây chính là lúc UNION ALL được sử dụng.

UNION ALL cũng kết hợp hai danh sách lại với nhau, nhưng giữ nguyên tất cả các giá trị, kể cả khi một cái tên xuất hiện nhiều hơn một lần.

Khi đó, danh sách cuối cùng sẽ bao gồm cả giá trị trùng lặp, ví dụ: Alice, Bob, Charlie, Bob, David và Eva.

Trong SQL, UNION ALL sẽ được viết như sau:

```
SELECT name FROM school_friends_table UNION ALL SELECT name FROM work_friends_table
```

Vậy là xong phần giải thích về UNION và UNION ALL.

Tiếp theo, chúng ta chuyển sang câu hỏi số 12.

Q.12 Query to Find Second Highest Salary in a Table

Làm thế nào để tìm mức lương cao thứ hai trong một bảng.

Truy vấn SQL để tìm mức lương cao thứ hai và những nhân viên nhận mức lương đó hoạt động theo một luồng có cấu trúc, sử dụng các truy vấn con lồng nhau.

Nó bắt đầu với truy vấn trong cùng, truy vấn này lấy ra mức lương cao nhất từ bảng nhân viên bằng câu lệnh

```
SELECT MAX(salary) FROM employee.
```

Ví dụ, nếu mức lương cao nhất là 100.000, thì giá trị này sẽ được sử dụng ở bước tiếp theo.

Ở truy vấn con thứ hai, hệ thống sẽ loại bỏ các mức lương bằng hoặc cao hơn mức lương cao nhất, bằng cách thực hiện

```
SELECT MAX(salary) FROM employee WHERE salary < (SELECT MAX(salary) FROM employee).
```

Bằng cách áp dụng hàm MAX lên các mức lương còn lại, truy vấn con này sẽ xác định mức lương cao thứ hai, trong trường hợp này có thể là 90.000.

Cuối cùng, truy vấn ngoài cùng sẽ được thực hiện để lấy tên và mức lương của nhân viên có mức lương cao thứ hai, bằng cách chạy:

```
SELECT name, salary FROM employee WHERE salary = (SELECT MAX(salary) FROM employee WHERE salary < (SELECT MAX(salary) FROM employee)).
```

Điều này có nghĩa là nếu có nhiều nhân viên cùng nhận mức lương cao thứ hai, thì truy vấn sẽ trả về tất cả những người đó.

Tóm lại, cách tiếp cận có cấu trúc này cho phép truy vấn xác định và hiển thị chính xác tên và mức lương của các nhân viên có thu nhập cao thứ hai một cách hiệu quả.

Vậy là xong cho câu hỏi này, tiếp theo chúng ta chuyển sang câu hỏi số 13.

Q.13 Views in SQL

Trong SQL, view giống như một bảng ảo. Nó không lưu trữ dữ liệu trên ổ đĩa, mà cho phép bạn xem dữ liệu từ một hoặc nhiều bảng theo một cách cụ thể.

Ý nghĩa của điều này là: giả sử bạn có một cơ sở dữ liệu sinh viên lưu trữ các thông tin như mã sinh viên, tên, địa chỉ, và có người chỉ muốn xem tên và địa chỉ sinh viên mà không cần xem mã sinh viên hoặc các thông tin khác.

Trong trường hợp này, bạn có thể tạo một view, ví dụ đặt tên view là detail_view.
Để làm điều này, câu lệnh bạn sử dụng sẽ là:

```
CREATE VIEW detail_view AS SELECT name, address FROM student_detail
```

Nếu bạn muốn thêm điều kiện, chẳng hạn chỉ lấy những bản ghi có ID nhỏ hơn 5, thì bạn thêm điều kiện WHERE ID < 5. Từ đó, một view sẽ được tạo ra.

Để xem nội dung của view, bạn chỉ cần thực hiện một câu lệnh đơn giản:
SELECT * FROM detail_view.

Khi đó, bạn sẽ thấy trên màn hình view chỉ hiển thị tên và địa chỉ.

Qua ví dụ này, bạn có thể thấy rằng view giúp giới hạn dữ liệu nhạy cảm và kiểm soát quyền truy cập của người dùng. Điều này đặc biệt hữu ích khi làm việc với các bảng lớn hoặc khi những người dùng khác nhau cần truy cập vào các phần dữ liệu khác nhau.

Tiếp theo, chúng ta chuyển sang câu hỏi số 14.

Q.14 How Can You Convert Text into Date Format?

Làm thế nào để chuyển văn bản (text) sang định dạng ngày (date).

Giả sử bạn có một chuỗi văn bản như 201122.

Để chuyển chuỗi (string) này sang định dạng ngày, bạn sử dụng hàm chuyển đổi chuỗi sang ngày (string to date function).

Câu lệnh sẽ có dạng: SELECT STR_TO_DATE(...)

Bên trong dấu ngoặc, bạn truyền vào chuỗi ngày và định dạng tương ứng, trong đó:

- %d đại diện cho ngày
- %m đại diện cho tháng
- %Y đại diện cho năm

SQL sẽ đọc chuỗi 27-10-2024 và hiểu nó là ngày 27 tháng 10 năm 2024 (October 27, 2024).

Tiếp theo, chúng ta chuyển sang câu hỏi số 15.

Q.15 What are Triggers in SQL

Trigger trong SQL là gì.

Hãy tưởng tượng bạn đang quản lý một thư viện, và mỗi lần có ai đó mượn hoặc trả sách, bạn muốn thực hiện các thao tác như cập nhật số lượng sách còn lại trong hệ thống. Nếu làm việc này thủ công cho từng giao dịch thì sẽ rất tốn thời gian và dễ xảy ra sai sót. Đây chính là lúc trigger trong SQL phát huy tác dụng.

Trigger giống như một hành động phản xạ, cho phép bạn thiết lập một hành động tự động sẽ được thực thi mỗi khi một sự kiện nhất định xảy ra trong cơ sở dữ liệu, chẳng hạn như thêm (insert), cập nhật (update) hoặc xóa (delete) dữ liệu trong một bảng.

Vậy khi nào trigger được kích hoạt?

Trigger thường được thiết lập để chạy trong ba trường hợp:

1. INSERT – khi dữ liệu mới được thêm vào bảng
2. UPDATE – khi dữ liệu hiện có trong bảng bị thay đổi
3. DELETE – khi dữ liệu bị xóa khỏi bảng

Ví dụ, giả sử bạn có một bảng trong cơ sở dữ liệu thư viện tên là Books. Mỗi lần có người mượn sách, bạn muốn:

- Giảm số lượng sách còn lại của cuốn sách đó
- Ghi lại giao dịch để theo dõi các sách đã được mượn

Bằng cách sử dụng trigger, bạn có thể làm cho các thao tác này diễn ra tự động mỗi khi có người mượn sách.

Giả sử bạn thiết lập một trigger để chạy khi một bản ghi mới được thêm vào bảng BorrowedBooks.

Câu lệnh sẽ là tạo một trigger có tên là update_book_stock. Tên này phản ánh đúng chức năng của trigger, đó là cập nhật số lượng sách.

Tiếp theo, bạn chỉ định trigger sẽ chạy sau khi có lệnh INSERT trên bảng BorrowedBooks. Điều này có nghĩa là mỗi khi một dòng mới được thêm vào bảng BorrowedBooks (tức là mỗi khi có sách được mượn), trigger sẽ được kích hoạt.

Từ khóa FOR EACH ROW có nghĩa là trigger sẽ áp dụng cho từng dòng dữ liệu được thêm vào. Nói cách khác, mỗi bản ghi mượn sách mới đều sẽ kích hoạt trigger.

Bên trong trigger, bạn thực hiện câu lệnh UPDATE bảng Books, giảm cột available_count đi 1 cho cuốn sách có book_id tương ứng. Ở đây, NEW.book_id đại diện cho mã sách vừa được mượn.

Tóm lại, mỗi lần có người mượn một cuốn sách, trigger này sẽ tự động giảm số lượng sách còn lại của cuốn đó trong bảng Books. Nhờ vậy, số lượng tồn kho trong bảng Books luôn được cập nhật chính xác mà không cần thao tác thủ công.

Đó là toàn bộ phần trình bày. Nếu bạn thích video này, hãy đăng ký kênh YouTube của Intellipaat. Cảm ơn bạn và hẹn gặp lại trong video tiếp theo.